BÁO CÁO LAB-02: Viết Device Driver & Quản lý File System

Embedded Linux Systems

Mục 1: Thông tin sinh viên và môi trường

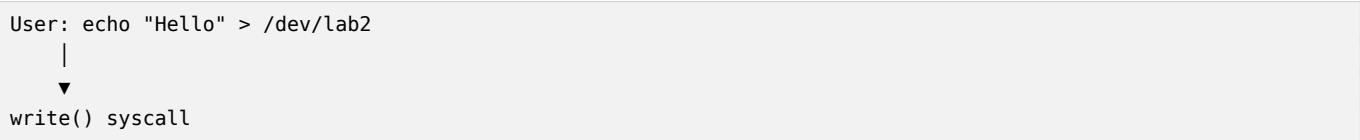
Mục	Nội dung
Họ và tên	Trần Đức Kiên
MSSV	SE183502
Ngày thực hiện	06/10/2026
Kiến trúc mục tiêu	ARMv7-A – Cortex-A9 (QEMU vexpress-a9)
Phiên bản kernel target	Linux 5.15.0
Phiên bản GCC toolchain	arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Vermagic module	`5.15.0 SMP mod_unload ARMv7 p2v8`

Mục 2: Sơ đồ kiến trúc driver

2.1. Luồng userspace → kernel space



2.2. Luồng dữ liệu khi write



```
    │
    ▼
VFS → lab2_write(file, buf [user ptr], len, offset)
    │
    ▼
mutex_lock(&lab2_mutex)      ← bảo vệ truy cập đồng thời
    │
    ▼
copy_from_user(device_buffer, buf, len)   ← kiểm tra + sao chép an toàn
    │
    ▼
buffer_len = len;  device_buffer[buffer_len] = '\0';
    │
    ▼
mutex_unlock(&lab2_mutex)
    │
    ├──► /proc/lab2_info   (lab2_proc_show đọc buffer_len, device_buffer khi cat)
    └──► /sys/.../last_data (last_data_show đọc device_buffer khi cat)
```

2.3. Cấu trúc /sys/class/lab2_class/lab2/

```
/sys/class/lab2_class/lab2/
├── buffer_len      (0444, r)  → sprintf(buf, "%d\n", buffer_len)
├── open_count      (0444, r)  → sprintf(buf, "%d\n", open_count)
├── last_data       (0444, r)  → sprintf(buf, "%s\n", device_buffer)
├── dev                        (tự sinh bởi device_create: "240:0")
├── uevent                     (tự sinh)
├── subsystem -> ../../../class/lab2_class
└── power/
```

Xác nhận từ log thực hành (xem Mục 3): `ls /sys/class/lab2_class/lab2/` trả về đúng `buffer_len last_data power uevent dev open_count subsystem`.

Mục 3: Log thực hành

> Toàn bộ log gốc, không chỉnh sửa, được đính kèm trong file nộp bài tại `logs/boot_log.txt`, `logs/driver_test.txt`, `logs/procfs_test.txt`, `logs/mtd_jffs2.txt`. Dưới đây là trích dẫn các đoạn quan trọng nhất làm bằng chứng.

3.1. Phần A — modinfo, insmod, lsmod, dmesg

```
$ modinfo lab2_driver.ko
filename:       lab2_driver.ko
version:        1.0
description:    LAB-02: Simple Character Device Driver
author:         Embedded Linux Lab
license:        GPL
srcversion:     863D9AAD089F03CDCB75EF0
depends:
name:           lab2_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
```

Boot log QEMU (trích `logs/boot_log.txt`):

```
[ LAB-02 ] Loading lab2_driver module...
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: /proc/lab2_info created
lab2_driver: loaded, major=240
```

```
[ LAB-02 ] lab2_driver loaded OK
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
```

Kết quả `lsmod` trong QEMU:

```
lab2_driver 16384 0 - Live 0x7f000000 (O)
```

Kết quả đọc/ghi `/dev/lab2` (trích `logs/driver_test.txt`):

```
[4] Writing to device...
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!
]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: device opened (count=2)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
  Read back: [Hello from userspace, LAB-02!]
```

`dmesg | grep lab2` trong QEMU (`logs/dmesg_lab2.txt`, sau một lần ghi rồi đọc `/dev/lab2`):

```
/ # echo "Hello dmesg test" > /dev/lab2; cat /dev/lab2
/ # dmesg | grep lab2
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: /proc/lab2_info created
lab2_driver: loaded, major=240
lab2_driver: device opened (count=1)
lab2_driver: received 17 bytes: [Hello dmesg test
lab2_driver: device closed
lab2_driver: device opened (count=2)
lab2_driver: sent 17 bytes to user
lab2_driver: device closed
```

Kernel log thể hiện đầy đủ vòng đời: `insmod` (init, tạo `/proc/lab2_info`, đăng ký major 240) → mỗi thao tác userspace là một chu trình open → write/read → release; `open_count` tăng dần qua các lần mở. Dòng `taints kernel` là cảnh báo bình thường khi nạp module out-of-tree.

3.2. Phần B — procfs/sysfs (trích `logs/procfs_test.txt`)

```
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
======================
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 10 bytes
Open count    : 8
Last data     : [Message_3
]

--- /proc/lab2_info (after write) ---
...
Data length   : 30 bytes
Open count    : 9
Last data     : [Embedded Linux Lab2 Test Data
]

--- sysfs attributes ---
buffer_len : 30
open_count : 9
```

```
last_data  : Embedded Linux Lab2 Test Data
=== procfs/sysfs Test PASSED ===
```

Quan sát: `/proc/lab2_info` và sysfs phản ánh đúng trạng thái `device_buffer` sau mỗi lần `write()`, chứng tỏ cả hai cơ chế cùng đọc chung một nguồn dữ liệu trong kernel.

3.3. Phần C — MTD & JFFS2 (trích `logs/mtd_jffs2.txt`, chạy trên HOST Ubuntu với sudo)

Phát hiện quan trọng: host đã có sẵn MTD thật tại `mtd0` (tên "BIOS"), nên `nandsim` được kernel gán vào `mtd1` ("NAND simulator partition 0") thay vì `mtd0`. Script Phần C được viết để tự động dò đúng thiết bị bằng cách parse `/proc/mtd`, thay vì giả định cứng `/dev/mtd0` như hướng dẫn gốc — đúng như lưu ý trong đề bài.

```
$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 02000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"

$ mtdinfo /dev/mtd1
Name:                           NAND simulator partition 0
Type:                           nand
Eraseblock size:                16384 bytes, 16.0 KiB
Amount of eraseblocks:          2048 (33554432 bytes, 32.0 MiB)
Minimum input/output unit size: 512 bytes
Device is writable:             true
```

Format và ghi JFFS2:

```
$ flash_erase /dev/mtd1 0 0
Erasing 32768 Kibyte @ 0 -- 100 % complete
$ mkfs.jffs2 --root=/tmp/jffs2_data --eraseblock=0x4000 --output=/tmp/rootfs.jffs2 --pad --no-cleanmarkers
$ nandwrite -p /dev/mtd1 /tmp/rootfs.jffs2
Writing data to block 0 at offset 0x0
```

Mount và đọc dữ liệu:

```
$ mount -t jffs2 /dev/mtdblock1 /mnt/jffs2
$ ls -la /mnt/jffs2/
hello.txt  info.txt  logs/
$ cat /mnt/jffs2/hello.txt
Hello from JFFS2!
$ cat /mnt/jffs2/logs/boot.log
Tue Oct  6 08:31:39 PM +07 2026
```

Kiểm chứng persistence (bằng chứng quan trọng nhất của Phần C):

```
$ echo "Written at $(date)" >> /mnt/jffs2/logs/boot.log
$ echo "New runtime file" > /mnt/jffs2/runtime.txt
$ umount /mnt/jffs2
$ mount -t jffs2 /dev/mtdblock1 /mnt/jffs2
$ cat /mnt/jffs2/logs/boot.log
Tue Oct  6 08:31:39 PM +07 2026
Written at Tue Oct  6 08:31:39 PM +07 2026      ← dữ liệu còn nguyên sau umount/mount
$ cat /mnt/jffs2/runtime.txt
New runtime file                                 ← file mới tạo cũng còn nguyên
```

Dump raw NAND và kiểm tra magic bytes JFFS2:

```
$ nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd1
$ hexdump -C /tmp/dump.bin | head -20
```

```
00000000  85 19 01 e0 31 00 00 00  1d d9 82 42 01 00 00 00  |....1......B....|
00000020  30 b3 8a 2c 15 da 69 f4  68 65 6c 6c 6f 2e 74 78  |0..,..i.hello.tx|
00000030  74 ff ff ff 85 19 02 e0  56 00 00 00 09 64 e7 62  |t.......V....d.b|
```

Magic bytes `85 19` xuất hiện ở đầu mỗi JFFS2 node (0x00, 0x24, 0x58...), đúng như lý thuyết (JFFS2_MAGIC_BITMASK = 0x1985). Tên file `hello.txt`, `info.txt` xuất hiện trực tiếp trong raw dump vì JFFS2 lưu tên file ngay trong dirent node.

3.4. Phần D — boot log QEMU auto-load

```
Run /sbin/init as init process
[ LAB-02 ] Starting system initialization...
[ LAB-02 ] Loading lab2_driver module...
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: /proc/lab2_info created
lab2_driver: loaded, major=240
[ LAB-02 ] lab2_driver loaded OK
mknod: /dev/lab2: File exists   ← devtmpfs đã tự tạo node trước, không ảnh hưởng chức năng
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
[ LAB-02 ] System initialization complete!
```

Module được load hoàn toàn tự động, không cần can thiệp thủ công sau khi boot; `test_driver.sh` và `test_procfs.sh` chạy PASS ngay sau đó. Toàn bộ output của hai script:

`test_driver.sh` (`logs/driver_test.txt`):

```
=== LAB-02: Device Driver Test ===
[1] Loading lab2_driver...
  (module already loaded)
[2] Loaded modules:
lab2_driver 16384 0 - Live 0x7f000000 (O)
[3] Creating device node...
  (node already exists)
crw-rw-rw-    1 root     0         240,   0 Oct  6 13:20 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!
]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: device opened (count=2)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
  Read back: [Hello from userspace, LAB-02!]
[6] Multiple write/read test...
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1
]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 10 bytes to user
Message_1
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2
]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
Message_2
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3
```

```
]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
Message_3
lab2_driver: device closed
[7] Kernel messages (dmesg):
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
=== Test PASSED ===
```

test_procfs.sh (logs/procfs_test.txt):

```
=== Test procfs & sysfs ===
  (module already loaded)
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
======================
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 10 bytes
Open count    : 8
Last data     : [Message_3
]
lab2_driver: device opened (count=9)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
======================
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 30 bytes
Open count    : 9
Last data     : [Embedded Linux Lab2 Test Data
]
--- sysfs attributes ---
buffer_len : 30
open_count : 9
last_data  : Embedded Linux Lab2 Test Data
=== procfs/sysfs Test PASSED ===
```

Mục 4: So sánh filesystem

Tiêu chí	ext4	JFFS2	cramfs
Writable (ghi runtime)	Có	Có	Không (read-only)
Wear leveling	Không (cần lớp dưới, vd FTL/eMMC controller)	Có, tích hợp sẵn	Không áp dụng

Compression	Không mặc định	Có (zlib/lzo)	Có (bắt buộc)
Boot/mount time	Nhanh (journal nhỏ, block-based)	Chậm với fs lớn (phải scan toàn bộ flash để dựng lại index khi mount)	Rất nhanh (không cần scan, chỉ đọc superblock)
Phù hợp thiết bị	eMMC, SD card, HDD (có FTL/block abstraction)	Raw NAND/NOR flash, cần ghi runtime	ROM/firmware tĩnh, không cần ghi
Chịu mất điện giữa chừng	Cần journal replay	Log-structured, tự phục hồi tốt	Không áp dụng (read-only)

Khuyến nghị cho thiết bị IoT có 32MB NAND flash, cần update firmware OTA:

Dùng kết hợp hai filesystem theo mô hình phân vùng:

- **cramfs hoặc SquashFS (read-only)** cho phân vùng chứa firmware/rootfs cơ bản

— nén tốt (tiết kiệm không gian trên NAND 32MB nhỏ), không sợ hỏng dữ liệu do mất điện vì không ghi.

- **JFFS2** cho phân vùng dữ liệu cấu hình/log cần ghi lúc runtime (vd:

`/data`, `/etc/config`) — có wear leveling và chịu được mất điện giữa chừng nhờ log-structured.

- Cơ chế OTA ghi đè toàn bộ image mới vào phân vùng rootfs (A/B partition

scheme) thay vì JFFS2 full-write, vì JFFS2 mount chậm dần khi filesystem lớn và gần đầy — không lý tưởng cho rootfs chính trên flash nhỏ 32MB.

Mục 5: Trả lời câu hỏi phân tích

Câu 1. Tại sao không dùng memcpy() giữa kernel buffer và user buffer?

Kernel và mỗi process userspace có không gian địa chỉ ảo (virtual address space) riêng biệt, được MMU ánh xạ độc lập qua bảng trang (page table) riêng cho từng process. Một con trỏ do userspace truyền vào syscall (ví dụ `buf` trong `write(fd, buf, len)`) là địa chỉ ảo *trong không gian của process gọi*, không có ý nghĩa trực tiếp trong ngữ cảnh kernel đang thực thi.

`memcpy()` chỉ thực hiện sao chép byte thuần túy, giả định cả hai con trỏ nguồn/đích đều **đã hợp lệ và truy cập được ngay lập tức** trong ngữ cảnh hiện tại. Nó không kiểm tra:

- Con trỏ có thực sự trỏ vào vùng nhớ của userspace (không phải địa chỉ kernel

bị truyền nhầm/cố ý — nguy cơ bảo mật: process có thể giả mạo con trỏ trỏ vào vùng kernel để đọc/ghi trái phép).

- Trang nhớ tương ứng đã được map vào RAM hay chưa (có thể đã bị swap-out,

cần page fault để load lại).

`copy_to_user()`/`copy_from_user()` giải quyết cả hai vấn đề: chúng gọi `access_ok()` để kiểm tra con tro nằm trong vùng địa chỉ userspace hợp lệ, và xử lý page fault một cách an toàn (có thể block để load trang) thông qua cơ chế exception table của kiến trúc. Nếu địa chỉ không hợp lệ, hàm trả về số byte chưa copy được (khác 0) thay vì crash, cho phép driver trả lỗi `-EFAULT` một cách sạch sẽ.

Nếu dùng memcpy() thay thế: nếu con trỏ user trỏ vào vùng chưa map hoặc không hợp lệ, CPU sẽ sinh page fault/exception mà kernel không có cơ chế xử lý tương ứng (không qua exception table của copy_*_user) → dẫn tới kernel oops hoặc panic, có thể crash toàn bộ hệ thống. Ngoài ra, nếu userspace cố tình truyền con trỏ trỏ vào vùng nhớ kernel, `memcpy()` sẽ vô tình đọc/ghi dữ liệu kernel ra ngoài — một lỗ hổng leak thông tin hoặc ghi đè bộ nhớ kernel nghiêm trọng.

Câu 2. Tại sao cần mutex bảo vệ device_buffer?

`device_buffer` và `buffer_len` là **tài nguyên toàn cục dùng chung** giữa mọi process đang mở `/dev/lab2` — không giống biến cục bộ trên stack của riêng mỗi lời gọi hệ thống. Khi nhiều process gọi `write()` đồng thời (trên hệ thống đa nhân/đa luồng, `smp=2` trong cấu hình QEMU của bài lab), các lệnh `copy_from_user()`, gán `buffer_len = len`, và `device_buffer[buffer_len] = '\0'` có thể bị xen kẽ

giữa hai process nếu không có khóa.

Race condition cụ thể nếu không có mutex: giả sử process A ghi chuỗi dài 20 byte, process B ghi chuỗi ngắn 5 byte gần như đồng thời. Trình tự có thể xảy ra:

1. A: `copy_from_user()` chép 20 byte vào `device_buffer`.
2. B chen vào: `copy_from_user()` chép 5 byte đầu của B, ghi đè lên 5 byte đầu của A trong `device_buffer`.
3. B: gán `buffer_len = 5`.
4. A (tiếp tục, bị trễ do scheduler): gán `buffer_len = 20` (ghi đè giá trị của B).

Kết quả: `buffer_len = 20` nhưng 5 byte đầu của `device_buffer` là dữ liệu của B còn 15 byte sau là của A — dữ liệu "Frankenstein" không khớp với bất kỳ lần ghi nào của cả A lẫn B. Process đọc lại sau đó (`read()`) sẽ nhận một chuỗi rác, sai lệch hoàn toàn so với những gì cả hai bên mong đợi — đây chính là hậu quả cổ điển của race condition trên tài nguyên chia sẻ không được đồng bộ hóa.

`mutex_lock()/mutex_unlock()` đảm bảo toàn bộ chuỗi thao tác (copy + cập nhật buffer_len + null-terminate) của một `write()` hoàn tất trọn vẹn trước khi `write()` khác được phép bắt đầu, loại bỏ hoàn toàn tình huống xen kẽ trên.

Câu 3. So sánh procfs và sysfs

	procfs (/proc)	sysfs (/sys)
Mục đích thiết kế ban đầu	Thông tin tiến trình (process info)	Mô hình hóa cây thiết bị/driver (device model, kobject)
Cấu trúc dữ liệu	Text tự do, không ràng buộc định dạng	Nguyên tắc "one value per file", có cấu trúc phân cấp theo kobject
API điển hình	seq_file, proc_create	sysfs_create_group, DEVICE_ATTR_* macro
Khả năng mở rộng	Dễ lạm dụng thành "bãi rác" thông tin không chuẩn hóa	Được kernel chuẩn hóa chặt chẽ, gắn với device model

Kernel documentation khuyến nghị dùng `/sys` thay vì thêm file mới vào `/proc` vì `/proc` ban đầu được thiết kế riêng cho thông tin tiến trình (process), và việc các driver tùy tiện thêm file thống kê/điều khiển vào đó qua nhiều năm đã biến `/proc` thành không gian tên thiếu tổ chức, không tuân theo quy ước nào về định dạng nội dung (mỗi driver tự in text theo ý mình). `sysfs` được thiết kế từ đầu gắn liền với device driver model (kobject, kset, bus, class), áp dụng quy ước "mỗi file sysfs chỉ chứa đúng một giá trị, dạng text đơn giản", giúp công cụ userspace (udev, systemd) parse một cách máy móc, nhất quán.

Áp dụng vào driver của bài lab:

- `/proc/lab2_info` phù hợp cho báo cáo thống kê tổng hợp dạng người-đọc

(nhiều trường thông tin trình bày cùng lúc dưới dạng báo cáo: tên driver, major, buffer size, open count, last data) — đúng tinh thần ban đầu của procfs là "thông tin chẩn đoán dạng text tự do", không cần máy móc khác parse riêng lẻ từng trường.

- `buffer_len`, `open_count`, `last_data` dưới `/sys/class/lab2_class/lab2/`

phù hợp hơn vì mỗi file chứa đúng một giá trị nguyên tử, dễ cho script/udev đọc riêng lẻ (`cat .../buffer_len`) mà không cần parse text, và gắn trực tiếp với kobject của device — đúng kiến trúc device model mà sysfs hướng tới.

Câu 4. Tại sao JFFS2 phải erase toàn bộ block trước khi ghi?

Đặc tính vật lý của NAND/NOR flash: mỗi ô nhớ (cell) chỉ có thể chuyển trạng thái bit theo một chiều khi ghi thông thường — thao tác ghi (program) chỉ có thể đổi bit từ `1` → `0`, không thể đổi ngược `0` → `1`. Muốn đưa bit về lại `1` (để ghi đè dữ liệu mới tùy ý), bắt buộc phải thực hiện thao tác **erase** — và erase trên NAND/NOR chỉ thực hiện được theo đơn vị **erase block** (ví dụ 16KB trong bài lab này với nandsim), không thể erase từng byte/sector nhỏ như ext4 ghi đè trực tiếp lên sector của ổ đĩa từ hoặc eMMC (nơi bộ điều khiển ổ đĩa/FTL đã ẩn đi giới hạn phần cứng này).

Vì vậy, muốn sửa một byte dữ liệu đã ghi trước đó trên raw flash, về lý thuyết phải erase nguyên khối chứa byte đó rồi ghi lại toàn bộ nội dung khối — rất tốn kém nếu làm trực tiếp (và erase còn có giới hạn số chu kỳ ~100.000 lần/khối, ảnh hưởng tuổi thọ).

Cơ chế log-structured của JFFS2 giải quyết vấn đề này bằng cách không bao giờ ghi đè (overwrite) tại chỗ: mỗi lần ghi/sửa dữ liệu, JFFS2 tạo một **node mới** (chứa dữ liệu + metadata) ở vùng trống phía sau log, đánh dấu node cũ là "obsolete" thay vì xóa ngay. Khi hết dung lượng trống, một tiến trình nền **garbage collection** sẽ chọn các erase block chứa nhiều node obsolete, di dời node còn hợp lệ sang nơi khác, rồi erase toàn bộ block đó để tái sử dụng. Cách tiếp cận ghi-tuần-tự này vừa tránh phải erase ngay lúc ghi (độ trễ thấp hơn), vừa tự nhiên phân bố đều số lần erase trên toàn bộ chip (wear leveling) vì không khối nào bị ghi/erase lặp lại nhiều hơn khối khác một cách có chủ đích.

Câu 5. rcS và xử lý lỗi insmod

Trong rcS của bài lab, nếu `insmod` thất bại, hệ thống **vẫn tiếp tục boot**: khối lệnh dùng `if [ $? -eq 0 ]; then ... else echo '... ERROR ...'; fi` — nhánh `else` chỉ in cảnh báo bằng `echo`, không có lệnh `exit` hay `reboot` nào dừng tiến trình init. Do BusyBox init chạy `rcS` như một script shell thông thường trong action `sysinit`, một lệnh con thất bại (exit code khác 0) **không** tự động làm dừng toàn bộ script (shell mặc định không bật `set -e`), nên các dòng phía sau (`echo '[ LAB-02 ] System initialization complete!'`, mở shell ở `askfirst`...) vẫn chạy bình thường.

Phân biệt lỗi nghiêm trọng (phải dừng boot) với lỗi không nghiêm trọng (tiếp tục với cảnh báo): về bản chất, BusyBox init không tự phân biệt — toàn bộ quyết định "dừng hay tiếp tục" nằm ở cách *người viết script rcS* xử lý mã trả về (`$?`) của từng lệnh. Nếu lỗi được coi là nghiêm trọng (ví dụ không mount được `/proc`, hệ thống không thể hoạt động), script nên chủ động gọi hành động dừng an toàn (vd in lỗi rõ ràng rồi rơi vào một `/bin/sh` cứu hộ, hoặc `reboot`). Nếu lỗi không ảnh hưởng tới khả năng boot tiếp (như module driver phụ không load được nhưng console/shell vẫn dùng được), script nên chỉ cảnh báo và tiếp tục — đúng như cách bài lab đã thiết kế cho `insmod lab2_driver`.

Cải thiện rcS để xử lý tốt hơn khi module load thất bại:

```
echo '[ LAB-02 ] Loading lab2_driver module...'
MODULE_LOAD_OK=0
if [ -f /lib/modules/5.15.0/lab2_driver.ko ]; then
    if insmod /lib/modules/5.15.0/lab2_driver.ko 2>/tmp/lab2_insmod.err; then
        echo '[ LAB-02 ] lab2_driver loaded OK'
        if mknod /dev/lab2 c 240 0 2>/dev/null; then
            chmod 666 /dev/lab2
            echo '[ LAB-02 ] /dev/lab2 created (major=240, minor=0)'
            MODULE_LOAD_OK=1
        else
            echo '[ LAB-02 ] WARNING: mknod /dev/lab2 failed (node co the da ton tai)'
            [ -c /dev/lab2 ] && MODULE_LOAD_OK=1
        fi
    else
        echo '[ LAB-02 ] ERROR: insmod lab2_driver failed!'
        echo "[ LAB-02 ] Chi tiet loi: $(cat /tmp/lab2_insmod.err)"
        echo '[ LAB-02 ] WARNING: tiep tuc boot khong co lab2_driver (non-fatal)'
    fi
else
    echo '[ LAB-02 ] WARNING: lab2_driver.ko not found, bo qua (non-fatal)'
fi

# Ghi trang thai vao /tmp de cac script/test sau co the kiem tra nhanh
echo "$MODULE_LOAD_OK" > /tmp/lab2_module_status
```

Cải tiến chính: bắt riêng lỗi `insmod` (ghi stderr ra file để xem nguyên nhân cụ thể — ví dụ vermagic không khớp) và lỗi `mknod` (phân biệt với trường hợp node đã tồn tại sẵn do devtmpfs), đồng thời ghi lại trạng thái cuối cùng vào `/tmp/lab2_module_status` để các script kiểm tra sau (`test_driver.sh`) có thể chủ động kiểm tra trước khi chạy, thay vì chạy mù và nhận lỗi `insmod` trùng lặp không rõ ràng.

Mục 6: Nhận xét

- Driver đi đủ các lớp của một character device thực tế: đăng ký major number, `file_operations`, tạo device node, xuất thông tin qua cả `/proc` (dạng text tổng hợp, dùng `seq_file`) và `/sys` (mỗi thuộc tính một file, gắn vào kobject của device).
- Ranh giới user/kernel phải luôn đi qua `copy_to_user()`/`copy_from_user()`; mọi trạng thái dùng chung (`device_buffer`, `open_count`) được bảo vệ bằng mutex vì nhiều process có thể mở `/dev/lab2` cùng lúc.
- JFFS2 trên NAND giả lập (`nandsim`) cho thấy rõ đặc tính flash: phải `flash_erase` trước khi ghi, dữ liệu ghi theo log và vẫn còn nguyên sau umount/mount lại. Thiết bị MTD phải được tìm theo tên trong `/proc/mtd` vì máy host đã có sẵn `mtd0` (BIOS) – `nandsim` nhận `mtd1`.
- Tích hợp vào BusyBox init biến driver thành một phần của hệ thống: module tự nạp khi boot qua `rcS`, lỗi nạp module được xử lý như lỗi không nghiêm trọng để hệ thống vẫn boot được.